



CHAPTER 05

Dictionary & Sets



Dictionary is a collection of **keys-value** pairs.

Syntax:

```
a = {
    "key": "value",
    "harry": "code",
    "marks": "100",
    "list": [1, 2, 9]
}
```

a (Dictionary)

Key	Value
"key"	"value"
"harry"	"code"
"marks"	"100"
"list"	[1, 2, 9]

Each key is unique and maps to its value.

```
print(a["key"])    # Output: "value"
print(a["list"])   # Output: [1, 2, 9]
```



Properties Of Python Dictionaries

- It is **unordered**.
- It is **mutable**.
- It is **indexed**.
- Cannot contain **duplicate keys**.

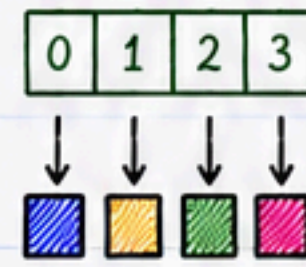
Unordered



Mutable



Indexed



No Duplicate Keys



Dictionary Methods

Consider the following dictionary.

```
a = {"name": "harry",
     "from": "india",
     "marks": [92, 98, 96]}
```

Dictionaries have many useful built-in methods to perform operations.



- a.items()**: Returns a list of (key,value) tuples. → `[('name', 'harry'), ('from', 'india'), ('marks', [92, 98, 96])]`
- a.keys()**: Returns a list containing dictionary's keys. → `['name', 'from', 'marks']`
- a.update({"friends": "rohan"})**: Updates the dictionary with supplied key-value pairs. → `a = {"name": "harry", "from": "india", "marks": [92, 98, 96], "friends": "rohan"}`
- a.get("name")**: Returns the value of the specified keys. → `'harry'`



More methods are available on docs.python.org



Dictionaries are very powerful and widely used in Python!





Sets In Python

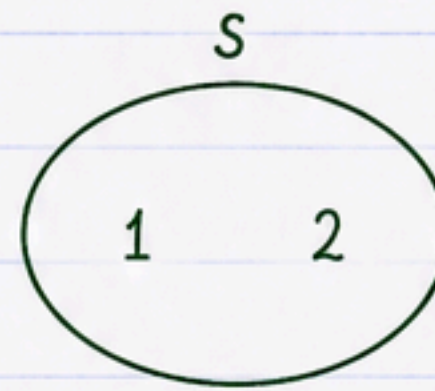


Set is a collection of **non-repetitive** elements.

```

s = set()    # no repetition allowed!
s.add(1)
s.add(2)     # or set = {1,2}

```



No duplicate elements allowed!



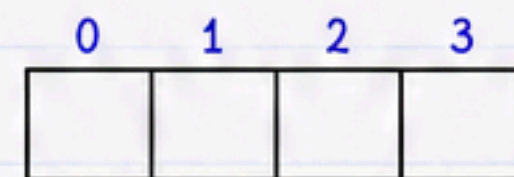
If you are a programming beginner without much knowledge of mathematical operations on sets, you can simply look at sets in python as data types containing **unique values**.

Properties Of Sets

- Sets are **unordered** $\Rightarrow$ Element's order doesn't matter
- Sets are **unindexed** $\Rightarrow$ Cannot access elements by index
- There is **no way to change** items in sets.
- Sets cannot contain **duplicate values**.

{ 3, 1, 8, 2 }

order doesn't matter



✗ Cannot use index



Immutable (cannot change items)

{ 1, 2, ~~2~~, 3 }

No duplicate values

Operations On Sets

`s = { 1, 8, 2, 3 }`

Consider the following set:

- len(s):** Returns 4, the length of the set

len(s) $\rightarrow$ 4



- s.remove(8):** Updates the set s and removes 8 from s.

s.remove(8) $\rightarrow$ { 1, 2, 3 }

8

- s.pop():** Removes an arbitrary element from the set and return the element removed.

s.pop() $\rightarrow$ (removed element)

s $\rightarrow$ { 1, 2, 3 }

{ 1, 8, 2, 3 }

{ 1, 2, 3 }

- s.clear():** empties the set s.

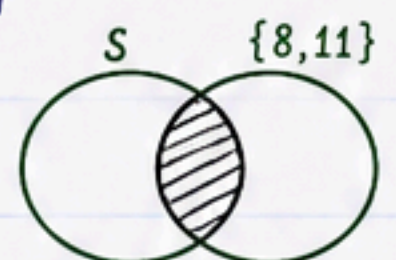
s.clear() $\rightarrow$ { }



- s.union({8,11}):** Returns a new set with all items from both sets.

s.union({8,11}) $\rightarrow$

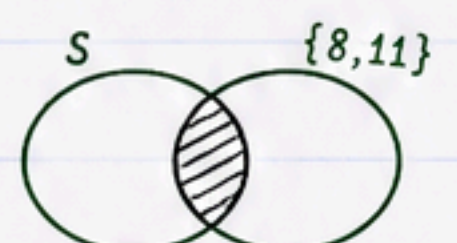
{ 1, 2, 3, 8, 11 }



- s.intersection({8,11}):** Returns a set which contains only item in both sets {8}.

s.intersection({8,11}) $\rightarrow$

{ 8 }



Sets store unique values and support powerful operations!





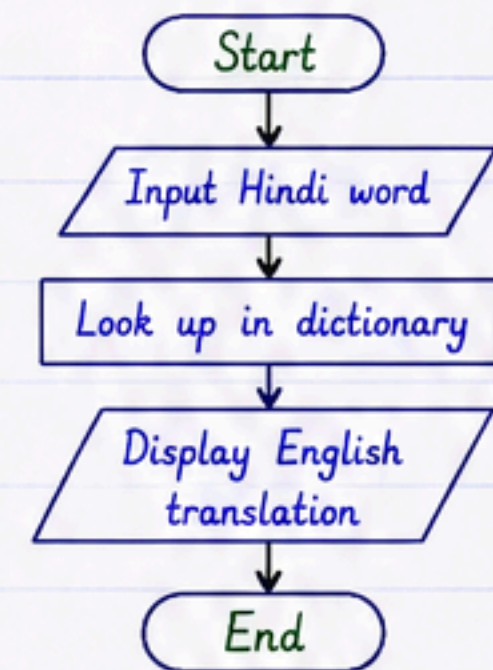
Practice Set



1. Write a program to create a **dictionary** of Hindi words with values as their English translation. Provide user with an option to **look it up!**

Example Dictionary

```
{  
  "namaste": "hello",  
  "pani": "water",  
  "kitab": "book"  
}
```



2. Write a program to input eight numbers from the user and display all the **unique numbers** (once).

Example

Input : 5 7 5 2 9 2 7 1
Output: 5 7 2 9 1

Use Set (no duplicates)

Input

5, 7, 5, 2,
9, 2, 7, 1

Set

5, 7, 2,
9, 1

3. Can we have a set with **18** (int) and **'18'** (str) as a value in it?

Yes!

Because int and str are different data types.

`s = {18, '18'}`

18 '18'

Both are different
(in value & type)



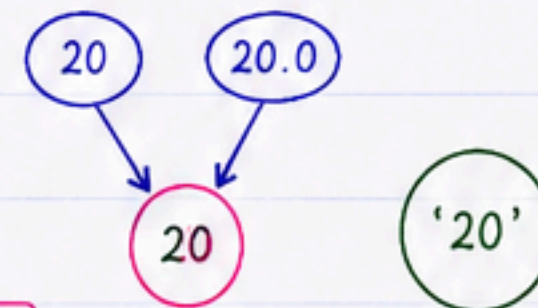
4. What will be the **length** of following set `s`:

```
s = set()  
s.add(20)  
s.add(20.0)  
s.add('20')
```

20 and 20.0 are considered the same in set (both are numeric and equal).
'20' (str) is different.

`s = {20, '20'}` $\Rightarrow$ `len(s) = 2`

Elements added



Final set
{20, '20'}
Length = 2

5. `s = {}`

What is the type of 's'?

It is an empty dictionary.

Type of 's' $\Rightarrow$ `dict`

Empty curly braces {} represent dictionary.

`s = {}`

Key	Value

6. Create an empty dictionary. Allow 4 friends to enter their **favorite language** as value and use key as their names. Assume that the names are **unique**.

Example

Name (Key)	Fav. Language (Value)
Harry	Python
Rohan	Java
Aman	C++
Sneha	JavaScript

Input from users.
2. Rohan ____
3. Aman ____
4. Sneha ____

7. If the **names** of 2 friends are same; what will happen to the program in problem 6?

Dictionary cannot have duplicate keys. The later value will overwrite the earlier value for that key.

Example

```
{'Harry': 'Python'}  
↓  
{'Harry': 'Java'}
```

Latest value, overwrites previous one

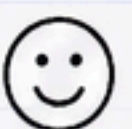
8. If **languages** of two friends are same; what will happen to the program in problem 6?

No problem!

Dictionary allows duplicate values. Both entries will be stored with the same value.

Example

```
{'Rohan': 'Java',  
'Sneha': 'Java'}
```



9. Can you change the values inside a list which is contained in set `S`?

`s = {8, 7, 12, "Harry", [1, 2]}`

No!

Because lists are mutable (changeable), and sets require immutable (unchangeable) elements.

List is mutable, so it cannot be an element of set

Set

8 7 12
"Harry" [1, 2]

